



Temat: Analizator wyników turnieju programistycznego.

Przygotuj program w języku Haskell, który przetwarza listę uczestników turnieju. Każdy uczestnik jest opisany krotką zawierającą imię i nazwisko, nazwę grupy oraz listę punktów z kolejnych zadań.

Zakres wymaganych umiejętności

- definiowanie aliasu typu dla krotki,
- przechowywanie danych jako listy krotek,
- użycie funkcji `map`, `filter`, `foldl` lub `sum`, `zip` albo `zipWith`,
- dopasowanie wzorców w funkcjach przyjmujących krotki,
- zwracanie wyników w postaci list i krotek,
- przygotowanie krótkiego sprawozdania wyjaśniającego działanie programu.

3. Wymagania funkcjonalne

Program ma zawierać co najmniej następujące funkcje:

Funkcja	Opis
<code>sumaPunktow</code>	oblicza sumę punktów jednego uczestnika
<code>sredniaPunktow</code>	oblicza średnią punktów jednego uczestnika
<code>wyniki</code>	zwraca listę par: imię i nazwisko oraz suma punktów
<code>zakwalifikowani</code>	zwraca uczestników z wynikiem co najmniej 60 punktów
<code>uczestnicyGrupy</code>	filtruje uczestników według nazwy grupy
<code>najlepszyWynik</code>	zwraca największą sumę punktów
<code>najlepsi</code>	zwraca wszystkich uczestników z najlepszym wynikiem
<code>raport</code>	zwraca listę napisów opisujących wynik każdego uczestnika

4. Kod startowy do pliku `projekt_turniej.hs`

```
-- Projekt domowy: Analizator wyników turnieju programistycznego
-- Przedmiot: Paradymaty programowania
-- Zakres: listy, krotki, funkcje map/filter/fold/zip
```

```
type Uczestnik = (String, String, [Int])
-- (imię i nazwisko, grupa, punkty z kolejnych zadań)
```

```
uczestnicy :: [Uczestnik]
uczestnicy =
  [ ("Anna Nowak", "INF3A", [18, 20, 17, 15])
  , ("Jan Kowalski", "INF3B", [12, 14, 10, 8])
  , ("Ola Mazur", "INF3A", [20, 19, 18, 20])
  , ("Piotr Lis", "INF3C", [15, 15, 16, 14])
  , ("Ewa Zielinska", "INF3B", [19, 18, 17, 18])
  ]
```

```
pobierzNazwe :: Uczestnik -> String
pobierzNazwe (nazwa, _, _) = nazwa
```

```
pobierzGrupe :: Uczestnik -> String
pobierzGrupe (_, grupa, _) = grupa
```

```
pobierzPunkty :: Uczestnik -> [Int]
pobierzPunkty (_, _, punkty) = punkty
```

```
-- TODO 1: oblicz sumę punktów jednego uczestnika
sumaPunktow :: Uczestnik -> Int
sumaPunktow uczestnik = undefined
```

```

-- TODO 2: oblicz srednia punktow jednego uczestnika
sredniaPunktow :: Uczestnik -> Double
sredniaPunktow uczestnik = undefined

-- TODO 3: zwroc liste par (nazwa uczestnika, suma punktow)
wyniki :: [Uczestnik] -> [(String, Int)]
wyniki xs = undefined

-- TODO 4: zostaw uczestnikow z wynikiem co najmniej 60 punktow
zakwalifikowani :: [Uczestnik] -> [Uczestnik]
zakwalifikowani xs = undefined

-- TODO 5: filtruj uczestnikow po grupie, np. "INF3A"
uczestnicyGrupy :: String -> [Uczestnik] -> [Uczestnik]
uczestnicyGrupy grupa xs = undefined

-- TODO 6: znajdz najlepszy wynik punktowy
najlepszyWynik :: [Uczestnik] -> Int
najlepszyWynik xs = undefined

-- TODO 7: zwroc wszystkich uczestnikow, ktorzy maja najlepszy wynik
najlepsi :: [Uczestnik] -> [Uczestnik]
najlepsi xs = undefined

-- TODO 8: przygotuj raport tekstowy
raport :: [Uczestnik] -> [String]
raport xs = undefined

-- Przykladowe testy w interpreterze:
-- sumaPunktow (head uczestnicy)
-- sredniaPunktow (head uczestnicy)
-- wyniki uczestnicy
-- zakwalifikowani uczestnicy
-- uczestnicyGrupy "INF3A" uczestnicy
-- najlepszyWynik uczestnicy
-- najlepsi uczestnicy
-- raport uczestnicy

```

5. Wymagania dotyczące oddawanego kodu

- Plik kodu powinien mieć nazwę nazwisko.hs.
- Każda funkcja musi mieć deklarację typu.
- Nie wolno usuwać danych startowych, ale można dopisać własne dane testowe.
- W kodzie należy użyć co najmniej trzech z funkcji: map, filter, foldl, foldr, zip, zipWith.
- Kod powinien uruchamiać się bez błędów po załadowaniu w WinHugs.
- W komentarzach należy dopisać przykładowe wywołania oraz spodziewane wyniki.

6. Sprawozdanie do projektu

Do kodu należy dołączyć krótkie sprawozdanie. Może mieć 1-2 strony. Sprawozdanie powinno zawierać odpowiedzi na poniższe pytania:

1. Jaki typ ma pojedynczy uczestnik i dlaczego wybrano krotkę?
2. Dlaczego dane wszystkich uczestników przechowywane są jako lista?
3. Które funkcje w programie wykorzystują map i w jakim celu?
4. Które funkcje w programie wykorzystują filter i jaki warunek logiczny stosują?
5. W jaki sposób obliczana jest suma oraz średnia punktów?
6. Czy w programie występuje ryzyko błędu dla pustej listy? W których funkcjach i jak można je ograniczyć?
7. Czym różni się lista od krotki w Haskellu?
8. Podaj dwa przykładowe wywołania funkcji z programu i wyjaśnij uzyskane wyniki.